==========

全项目代码详解

==========

项目：Olist 电商数据分析（SmartQuery） **文件说明**：逐文件、逐代码块解释每条代码的作用 **生成日期**：2026-07-22

一、 setup_db.py — 一键建库

文件位置：`setup_db.py`

1.1 导入与配置

```
import duckdb
import os
import sys
import time

DATA_DIR = os.path.join(os.path.dirname(__file__), "项目", "archive")
DB_PATH = os.path.join(os.path.dirname(__file__), "smartquery.duckdb")
```

解释： - 导入 **DuckDB**（嵌入式分析数据库引擎）、os/sys/time 标准库 - `DATA_DIR`：指向 `项目/archive/` 目录，存放原始 CSV 数据 - `DB_PATH`：生成的 DuckDB 数据库文件路径，放在项目根目录

1.2 核心表清单

```
TABLES = {
    "customers":     "olist_customers_dataset.csv",
    "orders":        "olist_orders_dataset.csv",
    "order_items":   "olist_order_items_dataset.csv",
    "order_payments":"olist_order_payments_dataset.csv",
    "products":      "olist_products_dataset.csv",
    "sellers":       "olist_sellers_dataset.csv",
    "order_reviews":    "olist_order_reviews_dataset.csv",
    "category_translation": "product_category_name_translation.csv",
}
```

解释： - 定义 8 张核心表的 **表名 → CSV 文件名的映射** - 覆盖 Olist 数据集的所有关键表：客户、订单、订单商品明细、支付、产品、卖家、评价、品类翻译

1.3 建表 SQL

```
SCHEMA_SQL = {
    "customers": """
        CREATE TABLE customers AS
        SELECT * FROM read_csv_auto('{path}')
    """,
```

```
    # ... 其他表结构相同
}
```

解释： - 使用 DuckDB 的 `read_csv_auto` 自动推断 CSV 的列名和类型 - `CREATE TABLE ... AS SELECT ...` 将 CSV 内容直接导入为内存表 - `{path}` 会被替换为实际 CSV 路径 - **作用：**比手动定义列类型更快，DuckDB 的自动推断在 Olist 数据集上表现良好

1.4 建索引

```
INDEX_SQL = [
    "CREATE INDEX idx_orders_customer ON orders(customer_id)",
    "CREATE INDEX idx_orders_date ON orders(order_purchase_timestamp)",
    "CREATE INDEX idx_items_order ON order_items(order_id)",
    "CREATE INDEX idx_items_product ON order_items(product_id)",
    "CREATE INDEX idx_customer_id ON customers(customer_id)",
    "CREATE INDEX idx_payments_order ON order_payments(order_id)",
    "CREATE INDEX idx_products_id ON products(product_id)",
]
```

解释： - 为经常 JOIN 和 WHERE 的列创建索引，加速后续分析查询 - `idx_orders_customer`：加速 orders 按客户 ID 关联客户表 - `idx_orders_date`：加速按月份/时间范围的查询 - `idx_items_order` 与 `idx_items_product`：加速订单商品明细与订单/产品的关联

1.5 创建分析视图

```
VIEWS_SQL = [
    # 视图1：订单+客户+金额宽表
    """CREATE OR REPLACE VIEW v_order_summary AS
    SELECT ... FROM orders o
    LEFT JOIN customers c ...
    LEFT JOIN order_items oi ...
    LEFT JOIN products p ...
    LEFT JOIN category_translation ct ...
    LEFT JOIN order_payments op ...""",

    # 视图2：用户维度聚合
    """CREATE OR REPLACE VIEW v_customer_metrics AS
    SELECT ... MAX(...) AS last_order_date,
           MIN(...) AS first_order_date,
           COUNT(DISTINCT order_id) AS order_count,
           SUM(oi.price) AS total_spend
    FROM orders o LEFT JOIN customers c ...
    LEFT JOIN order_items oi ...
    GROUP BY o.customer_id, ...""",
]
```

解释： - `v_order_summary`：将 6 张表通过 LEFT JOIN 拼成一张宽表。包含订单时间、客户信息、商品类别、支付方式等所有常用字段。注意：这是一个明细视图，一对多 JOIN 会导致行数膨胀 - `v_customer_metrics`：按客户聚合，计算每个客户的首单/末单时间、下单次数、总消费金额。这是 **RFM 分析**的基础数据

1.6 主流程 `main()`

```
def main():
    if "--reset" in sys.argv and os.path.exists(DB_PATH):
        os.remove(DB_PATH)
        print("已删除旧数据库")
    con = duckdb.connect(DB_PATH)
    start = time.time()
    # 1. 导入CSV
    for table_name, file_name in TABLES.items():
```

```
        file_path = os.path.join(DATA_DIR, file_name)
        if not os.path.exists(file_path):
            print(f"  [WARN]  文件不存在，跳过: {file_name}")
            continue
        con.execute(f"DROP TABLE IF EXISTS {table_name}")
        sql = SCHEMA_SQL[table_name].format(path=file_path.replace("\\", "/"))
        con.execute(sql)
        row_count = con.execute(f"SELECT COUNT(*) FROM {table_name}").fetchone()[0]
        print(f"  [OK] {table_name}: {row_count:,} 行")
```

解释： - `--reset` 参数可删除旧库重建 - 4 个步骤：[1/4] 导入CSV → [2/4] 创建索引 → [3/4] 创建视图 → [4/4] 验证 - 先 `DROP TABLE IF EXISTS` 确保幂等性 - 每张表导入后打印行数，方便核对

2.1 数据库路径解析

```
_ROOT = os.path.dirname(os.path.dirname(__file__))
_DB_CANDIDATES = [
    os.path.join(_ROOT, "smartquery.duckdb"),
    os.path.join(os.path.dirname(_ROOT), "smartquery.duckdb"),
]
DB_PATH = next((p for p in _DB_CANDIDATES if os.path.exists(p)), None)
if DB_PATH is None:
    raise FileNotFoundError("找不到 smartquery.duckdb。请放到项目根目录或 E:/workspace/ 下。")
```

解释： - `_ROOT` 从 `scripts/data_cleaning.py` 向上两级回到项目根目录 - 提供两个候选路径：项目根目录下 或 `E:/workspace/` 下 - 用 `next()` + 生成器表达式自动找到第一个存在的数据库文件

2.2 [1/5] v_valid_orders — 有效订单视图

```
con.execute("""
    CREATE OR REPLACE VIEW v_valid_orders AS
    SELECT * FROM orders
    WHERE order_status NOT IN ('canceled', 'unavailable')
""")
cnt = con.execute("SELECT COUNT(*) FROM v_valid_orders").fetchone()[0]
print(f"  有效订单: {cnt:,} / 99,441")
```

解释： - **问题**：原始 orders 表包含 1,234 单 canceled/unavailable，这些不是真正的交易 - **处理**：排除这两种状态，保留有效订单供分析（约 98,207 单）

2.3 [2/5] v_unique_customers — 去重客户视图

```
con.execute("""
    CREATE OR REPLACE VIEW v_unique_customers AS
    SELECT customer_id, customer_unique_id, customer_city, customer_state
    FROM (
        SELECT *, ROW_NUMBER() OVER (
            PARTITION BY customer_unique_id ORDER BY customer_id
        ) as rn
        FROM customers
    ) t
    WHERE rn = 1
""")
cnt = con.execute("SELECT COUNT(*) FROM v_unique_customers").fetchone()[0]
print(f"  去重后客户数: {cnt:,} (原 99,441, 去重 {99_441 - cnt:,})")
```

解释： - **问题**：`customer_unique_id` 存在 3,345 条重复。同一个真实客户（如搬家）在多个地址有不同 `customer_id` - **处理**：使用 `ROW_NUMBER() OVER (PARTITION BY customer_unique_id ORDER BY customer_id)` 为每个唯一客户编号，保留最早出现的记录（rn=1） - **结果**：96,096 个唯一客户

2.4 [3/5] v_order_delivery — 订单送达视图

```
con.execute("""
    CREATE OR REPLACE VIEW v_order_delivery AS
    SELECT
        order_id, order_status, order_purchase_timestamp,
        CASE WHEN order_delivered_customer_date IS NOT NULL THEN 1 ELSE 0 END as is_delivered,
        CASE WHEN order_approved_at IS NOT NULL THEN 1 ELSE 0 END as is_approved,
        CASE
            WHEN order_delivered_customer_date <= order_estimated_delivery_date THEN 1
            WHEN order_delivered_customer_date IS NOT NULL THEN 0
            ELSE NULL
        END as is_on_time,
        order_estimated_delivery_date, order_delivered_customer_date,
        DATEDIFF('day', order_purchase_timestamp, order_delivered_customer_date) as delivery_days
    FROM orders
    WHERE order_status = 'delivered'
""")
```

解释： - 仅筛选 `order_status = 'delivered'`（已送达的订单才有时效评价意义） - `is_delivered`：是否有送达日期 - `is_approved`：是否通过审批 - `is_on_time`：核心判断——实际送达 <= 预估送达日期则为准时(1)，否则延误(0) - `delivery_days`：从下单到送达的实际天数

2.5 [4/5] v_analysis_base — 明细宽表

```
con.execute("""
    CREATE OR REPLACE VIEW v_analysis_base AS
    SELECT
        o.order_id, o.customer_id,
        c.customer_unique_id, c.customer_city, c.customer_state,
        o.order_status, o.order_purchase_timestamp,
        o.order_approved_at, o.order_delivered_customer_date, o.order_estimated_delivery_date,
        STRFTIME(o.order_purchase_timestamp, '%Y-%m') as order_month,
        STRFTIME(o.order_purchase_timestamp, '%Y') as order_year,
        oi.product_id, p.product_category_name, ct.product_category_name_english,
        oi.price, oi.freight_value, (oi.price + oi.freight_value) as total_item_value,
        op.payment_type, op.payment_installments, op.payment_value,
        r.review_score,
        DATEDIFF('day', o.order_purchase_timestamp, o.order_delivered_customer_date) as actual_delivery_days,
        DATEDIFF('day', o.order_purchase_timestamp, o.order_estimated_delivery_date) as estimated_delivery_days,
        CASE WHEN o.order_delivered_customer_date IS NOT NULL
             THEN DATEDIFF(...) - DATEDIFF(...) ELSE NULL END as delay_days,
        CASE WHEN o.order_delivered_customer_date IS NOT NULL
              AND DATEDIFF(...) > DATEDIFF(...) THEN 1 ELSE 0 END as is_delayed
    FROM v_valid_orders o
    LEFT JOIN v_unique_customers c ON o.customer_id = c.customer_id
    LEFT JOIN order_items oi ON o.order_id = oi.order_id
    LEFT JOIN products p ON oi.product_id = p.product_id
    LEFT JOIN category_translation ct ON p.product_category_name = ct.product_category_name
    LEFT JOIN order_payments op ON o.order_id = op.order_id
    LEFT JOIN order_reviews r ON o.order_id = r.order_id
""")
```

解释： - 这是项目的**核心明细视图**，将 5 张清洗/原始表通过 LEFT JOIN 拼合 - 使用 `v_valid_orders`（排除脏订单）和 `v_unique_customers`（去重客户） - 新增派生字段：order_month / order_year / total_item_value / delay_days / is_delayed - **重要**：由于 LEFT JOIN 了 order_items（一对多）、order_payments（一对多），行数会膨胀。**统计订单级指标（延误率、差评率）时不能用这个视图直接 COUNT**

2.6 [5/5] v_order_delay — 订单级延误视图

```
con.execute("""
    CREATE OR REPLACE VIEW v_order_delay AS
    SELECT
```

```
        order_id,
        ANY_VALUE(customer_id) AS customer_id,
        ANY_VALUE(customer_unique_id) AS customer_unique_id,
        ANY_VALUE(customer_city) AS customer_city,
        ANY_VALUE(customer_state) AS customer_state,
        ANY_VALUE(order_status) AS order_status,
        ANY_VALUE(order_month) AS order_month,
        ANY_VALUE(actual_delivery_days) AS actual_delivery_days,
        ANY_VALUE(estimated_delivery_days) AS estimated_delivery_days,
        ANY_VALUE(delay_days) AS delay_days,
        ANY_VALUE(is_delayed) AS is_delayed,
        AVG(review_score) AS review_score
    FROM v_analysis_base
    WHERE delay_days IS NOT NULL
    GROUP BY order_id
"""
```

解释： - **核心理念**：从明细宽表按 `order_id` 聚合，将多行归为一行 - 使用 `ANY_VALUE()`：对于客户、城市、状态等每个订单内相同的字段，任取一个值即可 - 使用 `AVG(review_score)`：如果一个订单有多个评价（一对多），取平均分 - `WHERE delay_days IS NOT NULL`：只保留有送达日期的订单 - **这是整个项目的关键视图**——所有**订单级指标**（延误率、差评率、容忍度曲线）都基于此视图，确保口径正确

3.1 GMV 月度趋势

```
SELECT order_month, COUNT(DISTINCT order_id) as order_count,
    ROUND(SUM(total_item_value), 2) as gmv,
    ROUND(AVG(total_item_value), 2) as avg_order_value,
    ROUND(SUM(total_item_value) / COUNT(DISTINCT order_id), 2) as aov_by_order
FROM v_analysis_base GROUP BY order_month ORDER BY order_month;
```

解释： - 按 `order_month`（如 “2017-01”）分组统计 - `order_count`：用 `COUNT(DISTINCT order_id)` 避免多行膨胀 - `gmv`：总销售额 = SUM(price + freight) - `aov_by_order`：真实客单价 = GMV / DISTINCT 订单数 - **发现**：GMV 从月均 15 万增长至 110 万，增长约 7 倍

3.2 各州销售额排名

```
SELECT customer_state, COUNT(DISTINCT order_id) as order_count,
    ROUND(SUM(total_item_value), 2) as total_sales,
    ROUND(AVG(total_item_value), 2) as avg_order_value,
    ROUND(SUM(total_item_value) * 100.0 / SUM(SUM(total_item_value)) OVER(), 2) as sales_pct
FROM v_analysis_base GROUP BY customer_state ORDER BY total_sales DESC;
```

解释： - 使用 `SUM(SUM(total_item_value)) OVER()` 窗口函数计算总 GMV 作为分母，实现 “部分占总体的百分比” - **发现**：SP（圣保罗州）以 597 万 GMV 遥遥领先，占总量的 40%

3.3 支付方式分布

```
SELECT payment_type, COUNT(*) as transaction_count,
    ROUND(SUM(payment_value), 2) as total_amount,
    ROUND(AVG(payment_value), 2) as avg_amount, ROUND(AVG(payment_installments), 1) as avg_installments
FROM v_analysis_base WHERE payment_type IS NOT NULL
GROUP BY payment_type ORDER BY transaction_count DESC;
```

解释： - **发现**：信用卡占 74%，Boleto（巴西本地支付）占 20%

3.4 商品类别销售额 Top15

```
SELECT product_category_name_english, COUNT(DISTINCT order_id) as order_count,
    ROUND(SUM(price), 2) as total_sales, ROUND(AVG(price), 2) as avg_price,
    ROUND(SUM(price) * 100.0 / SUM(SUM(price)) OVER(), 2) as sales_pct
FROM v_analysis_base WHERE product_category_name_english IS NOT NULL
GROUP BY product_category_name_english ORDER BY total_sales DESC LIMIT 15;
```

解释： - **发现**：健康美容（130万）、手表礼品（125万）、床浴用品（111万）居前三

3.5 评分分布

```
SELECT review_score, COUNT(*) as review_count,
    ROUND(COUNT(*) * 100.0 / SUM(COUNT(*)) OVER(), 2) as pct
FROM v_analysis_base WHERE review_score IS NOT NULL
GROUP BY review_score ORDER BY review_score;
```

解释： - **发现**：评分呈 U 型分布——1 分和 5 分最多，中间评分较少，表明客户倾向于两极评价

3.6 月 GMV 环比变化率

```
WITH monthly_gmv AS (
    SELECT order_month, ROUND(SUM(total_item_value), 2) as gmv
    FROM v_analysis_base GROUP BY order_month
)
SELECT order_month, gmv, LAG(gmv) OVER (ORDER BY order_month) as prev_month_gmv,
    ROUND((gmv - LAG(gmv) OVER (ORDER BY order_month)) / LAG(gmv) OVER (ORDER BY order_month) * 100, 2) as mom_change_pct
FROM monthly_gmv ORDER BY order_month;
```

解释： - `LAG(gmv)` 窗口函数获取上个月 GMV - `mom_change_pct`：环比增长率 = (本月 - 上月) / 上月 * 100 - **发现**：2017 年 11 月因黑五效应达到峰值 122 万

4.1 整体漏斗

```
SELECT "下单" as stage, COUNT(*) as cnt, 100.0 as pct FROM orders
UNION ALL
SELECT "审批通过", COUNT(*), ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM orders), 2)
FROM orders WHERE order_approved_at IS NOT NULL
UNION ALL
SELECT "已发货", COUNT(*), ... FROM orders WHERE order_delivered_carrier_date IS NOT NULL
UNION ALL
SELECT "已送达", COUNT(*), ... FROM orders WHERE order_delivered_customer_date IS NOT NULL
UNION ALL
SELECT "已评价", COUNT(DISTINCT order_id), ... FROM order_reviews;
```

解释： - 用 `UNION ALL` 逐阶段拼接结果 - 每个阶段的 pct 以全量 orders 为基准 - 评价阶段用 `COUNT(DISTINCT order_id)` 而非 `COUNT(*)`

4.2 分月漏斗转化率

```
WITH monthly_funnel AS (
    SELECT STRFTIME(order_purchase_timestamp, "%Y-%m") as month,
        COUNT(*) as total_orders,
        SUM(CASE WHEN order_approved_at IS NOT NULL THEN 1 ELSE 0 END) as approved,
        SUM(CASE WHEN order_delivered_carrier_date IS NOT NULL THEN 1 ELSE 0 END) as shipped,
        SUM(CASE WHEN order_delivered_customer_date IS NOT NULL THEN 1 ELSE 0 END) as delivered
    FROM orders GROUP BY month
)
SELECT month, total_orders, approved, ROUND(approved*100.0/total_orders,1) as approval_rate,
    shipped, ROUND(shipped*100.0/total_orders,1) as ship_rate,
    delivered, ROUND(delivered*100.0/total_orders,1) as delivery_rate
FROM monthly_funnel ORDER BY month;
```

解释： - CTE 按月聚合审批/发货/送达的数量（SUM + CASE WHEN） - 观察转化率是否有季节性变化

4.3 差评 vs 好评订单的对比

```
SELECT CASE WHEN r.review_score >= 4 THEN "好评(4-5)"
            WHEN r.review_score <= 2 THEN "差评(1-2)" ELSE "中评(3)" END as review_group,
    COUNT(DISTINCT o.order_id) as order_count,
    ROUND(AVG(r.review_score), 2) as avg_score,
    ROUND(AVG(DATEDIFF("day", o.order_purchase_timestamp, o.order_delivered_customer_date)), 1) as avg_delivery_days,
    ROUND(AVG(oi.price), 2) as avg_price,
    ROUND(SUM(CASE WHEN o.order_delivered_customer_date > o.order_estimated_delivery_date THEN 1 ELSE 0 END)
        * 100.0 / COUNT(DISTINCT o.order_id), 1) as late_delivery_pct
FROM orders o JOIN order_reviews r ON o.order_id = r.order_id
LEFT JOIN order_items oi ON o.order_id = oi.order_id
WHERE o.order_status = "delivered" GROUP BY review_group;
```

解释： - 将订单分为三组：好评(4-5分)、中评(3分)、差评(1-2分) - 对比三组的平均配送天数、平均价格、延误率 - **核心发现**：差评组的延误率远高于好评组，暗示延误与差评正相关

5.1 v_rfm_raw — 计算 RFM 原始值

```
CREATE OR REPLACE VIEW v_rfm_raw AS
SELECT c.customer_unique_id, c.customer_city, c.customer_state,
    DATEDIFF("day", MAX(o.order_purchase_timestamp), "2018-10-18") as recency,
    COUNT(DISTINCT o.order_id) as frequency,
    ROUND(SUM(oi.price), 2) as monetary
FROM v_unique_customers c
JOIN v_valid_orders o ON c.customer_id = o.customer_id
LEFT JOIN order_items oi ON o.order_id = oi.order_id
GROUP BY c.customer_unique_id, c.customer_city, c.customer_state;
```

解释： - **R (Recency)**：最近一次下单距今的天数（以 2018-10-18 为参考），越小越好 - **F (Frequency)**：下单次数，越大越好 - **M (Monetary)**：消费总额，越大越好

5.2 v_rfm_scored — R/F/M 各分 5 档

```
CREATE OR REPLACE VIEW v_rfm_scored AS
SELECT *,
    CASE WHEN recency <= (SELECT PERCENTILE_CONT(0.2) WITHIN GROUP (ORDER BY recency) FROM v_rfm_raw) THEN 5
         WHEN recency <= (SELECT PERCENTILE_CONT(0.4) ...) THEN 4 ... ELSE 1 END as r_score,
    CASE WHEN frequency >= (SELECT PERCENTILE_CONT(0.8) WITHIN GROUP (ORDER BY frequency) FROM v_rfm_raw) THEN 5
         ... ELSE 1 END as f_score,
    CASE WHEN monetary >= (SELECT PERCENTILE_CONT(0.8) WITHIN GROUP (ORDER BY monetary) FROM v_rfm_raw) THEN 5
         ... ELSE 1 END as m_score
FROM v_rfm_raw;
```

解释： - 使用 PERCENTILE_CONT() 百分位数函数，按数据实际分布划分档次 - R 分数：Recency 越小（近期消费）→ 分数越高 - F/M 分数：Frequency/Monetary 越大 → 分数越高 - **等频分箱**策略：每个分数段约占总体的 20%

5.3 v_rfm_segments — 用户分群

```
CREATE OR REPLACE VIEW v_rfm_segments AS
SELECT *, (r_score + f_score + m_score) as rfm_total,
    CASE WHEN (r_score >= 4 AND f_score >= 4 AND m_score >= 4) THEN "重要价值用户"
         WHEN (r_score >= 4 AND f_score >= 4) THEN "重要发展用户"
         WHEN (r_score >= 4 AND m_score >= 4) THEN "重要保持用户"
         WHEN (f_score >= 4 AND m_score >= 4) THEN "重要挽留用户"
         WHEN (r_score >= 3) THEN "一般价值用户"
         WHEN (f_score >= 3) THEN "一般发展用户"
```

```
        WHEN (m_score >= 3) THEN "一般保持用户"
        ELSE "流失风险用户" END as segment
FROM v_rfm_scored;
```

解释： - **重要价值用户**（R>=4, F>=4, M>=4）：最优质客户 - **重要发展用户**（R>=4, F>=4）：频次高但金额不高 - **重要保持用户**（R>=4, M>=4）：金额高但频次不高 - **重要挽留用户**（F>=4, M>=4）：频次高但很久没消费 - **流失风险用户**：所有分数都不高

5.4-5.5 各分群统计和各州对比

```
-- 5.4 各分群统计
SELECT segment, COUNT(*) as customer_count,
    ROUND(COUNT(*) * 100.0 / SUM(COUNT(*)) OVER(), 2) as customer_pct,
    ROUND(AVG(monetary), 2) as avg_spend, ROUND(AVG(recency), 1) as avg_recency_days,
    ROUND(SUM(monetary), 2) as total_spend,
    ROUND(SUM(monetary) * 100.0 / SUM(SUM(monetary)) OVER(), 2) as spend_pct
FROM v_rfm_segments GROUP BY segment ORDER BY total_spend DESC;

-- 5.5 各州 RFM 均值对比
SELECT customer_state, COUNT(*) as customer_count,
    ROUND(AVG(recency), 1) as avg_recency, ROUND(AVG(frequency), 2) as avg_frequency,
    ROUND(AVG(monetary), 2) as avg_spend,
    ROUND(AVG(r_score), 2) as avg_r, ROUND(AVG(f_score), 2) as avg_f,
    ROUND(AVG(m_score), 2) as avg_m, ROUND(AVG(rfm_total), 2) as avg_rfm
FROM v_rfm_segments GROUP BY customer_state
HAVING COUNT(*) > 100 ORDER BY avg_rfm DESC;
```

解释： - 5.4 统计每个分群的客户数、占比、平均消费、总消费占比 - 5.5 过滤样本量过小的州（HAVING > 100），高价值用户集中在 SP/RJ/MG

6.1 延误率概览

```
SELECT ROUND(100.0 * SUM(is_delayed) / COUNT(*), 2) as delay_rate,
    ROUND(AVG(CASE WHEN is_delayed = 1 THEN delay_days END), 1) as avg_delay_days,
    MAX(delay_days) as max_delay, COUNT(*) as total_orders
FROM v_order_delay;
```

解释： - 基于 `v_order_delay`（订单级去重），确保分母是订单数而非明细行数 - **结果**：延误率 6.77%，平均延误 10.6 天

6.2 延误 vs 准点评分对比

```
SELECT is_delayed, COUNT(*) as order_count, ROUND(AVG(review_score), 2) as avg_score,
    ROUND(100.0 * SUM(CASE WHEN review_score <= 2 THEN 1 ELSE 0 END) / COUNT(*), 2) as bad_review_rate
FROM v_order_delay WHERE review_score IS NOT NULL GROUP BY is_delayed;
```

解释： - `bad_review_rate`：差评率 = 评分 <= 2 的订单占比 - **核心发现**：准点组差评率 9.23%，延误组差评率 62.36%，约 **6.8 倍差距**

6.3 延误天数分箱——容忍度阈值

```
SELECT CASE WHEN delay_days <= 0 THEN "准时/提前"
            WHEN delay_days BETWEEN 1 AND 3 THEN "1-3天"
            WHEN delay_days BETWEEN 4 AND 7 THEN "4-7天"
            WHEN delay_days BETWEEN 8 AND 15 THEN "8-15天"
            ELSE "16天以上" END as delay_range,
    COUNT(*) as order_count, ROUND(AVG(review_score), 2) as avg_score,
    ROUND(100.0 * SUM(CASE WHEN review_score <= 2 THEN 1 ELSE 0 END) / COUNT(*), 2) as bad_rate,
    ROUND(100.0 * SUM(CASE WHEN review_score >= 4 THEN 1 ELSE 0 END) / COUNT(*), 2) as good_rate
```

```
FROM v_order_delay WHERE review_score IS NOT NULL
GROUP BY delay_range ORDER BY MIN(delay_days);
```

解释： - 将延误天数分为 5 个区间 - **最关键发现**： - 准时/提前：差评率 9.23% - 延误 1-3 天：差评率 32.18% - 延误 4-7 天：差评率 **67.56%**（陡升！） - 延误 8-15 天：差评率 79.95% - 延误 16 天+：差评率 78.14% - **结论**：客户容忍度阈值约在 **3 天**，超过后差评率从 ~32% 陡升至 ~68%

6.4 同品类分层对比（控制混淆变量）

```
WITH order_category AS (
    SELECT DISTINCT order_id, product_category_name_english AS category,
           is_delayed, review_score
    FROM v_analysis_base
    WHERE review_score IS NOT NULL AND product_category_name_english IS NOT NULL AND delay_days IS NOT NULL
)
SELECT category, COUNT(*) as order_count,
    ROUND(AVG(CASE WHEN is_delayed = 1 THEN review_score END), 2) as delayed_avg,
    ROUND(AVG(CASE WHEN is_delayed = 0 THEN review_score END), 2) as ontime_avg,
    ROUND(AVG(CASE WHEN is_delayed = 1 THEN review_score END)
        - AVG(CASE WHEN is_delayed = 0 THEN review_score END), 2) as score_diff
FROM order_category GROUP BY category
HAVING COUNT(*) >= 200 ORDER BY score_diff ASC LIMIT 15;
```

解释： - **目的**：排除“不同品类质量不同”对评分的干扰 - CTE 按 `order_id + category` 去重（避免评价一对多膨胀） - `score_diff` = 延误评分 - 准点评分（负数表示延误更差） - **结果**：所有品类 score_diff 均为负，延误的负面影响仍然显著

6.5 各品类延误率排名

```
WITH order_category AS (
    SELECT DISTINCT order_id, product_category_name_english AS category, is_delayed, delay_days
    FROM v_analysis_base
    WHERE product_category_name_english IS NOT NULL AND delay_days IS NOT NULL
)
SELECT category, COUNT(*) as total,
    ROUND(100.0 * SUM(is_delayed) / COUNT(*), 2) as delay_rate,
    ROUND(AVG(CASE WHEN is_delayed = 1 THEN delay_days END), 1) as avg_delay_when_late
FROM order_category GROUP BY category
HAVING COUNT(*) >= 200 ORDER BY delay_rate DESC LIMIT 15;
```

解释： - 音响、家居用品、办公家具等品类延误率较高

八、Notebook 代码解析

8.1 00_data_exploration.ipynb — 数据探索

8.1.1 导入与环境

```
import duckdb, pandas as pd, numpy as np
import matplotlib.pyplot as plt, seaborn as sns
import warnings; warnings.filterwarnings("ignore")
plt.rcParams["font.sans-serif"] = ["SimHei", "Microsoft YaHei", "DejaVu Sans"]
plt.rcParams["axes.unicode_minus"] = False
con = duckdb.connect("smartquery.duckdb")
tables = con.execute("SHOW TABLES").fetchall()
for t in tables:
    cnt = con.execute(f"SELECT COUNT(*) FROM {t[0]}").fetchone()[0]
    print(f"  {t[0]}: {cnt:,} 行")
```

8.1.2 缺失值检查函数

```
def check_missing(con, table_name):
    cols = con.execute("SELECT column_name FROM information_schema.columns "
                       "WHERE table_name='" + table_name + "'").fetchdf()
    col_list = cols["column_name"].tolist()
    results = []
    for col in col_list:
        total = con.execute(f"SELECT COUNT(*) FROM {table_name}").fetchone()[0]
        non_null = con.execute(f"SELECT COUNT({col}) FROM {table_name}").fetchone()[0]
        missing = total - non_null
        pct = round(missing / total * 100, 2) if total > 0 else 0
        results.append({"table": table_name, "column": col, "missing": missing, "pct": pct})
    return pd.DataFrame(results)
```

解释：遍历每一列，用 `COUNT(col)` 排除 NULL 的原理计算缺失值和百分比。特别检查 orders 表三个时间字段的缺失情况。

8.1.3 EDA 可视化（六子图面板）

```
fig, axes = plt.subplots(2, 3, figsize=(18, 12))
# 子图1: 订单数随月份折线
# 子图2: 价格分布直方图（去除99分位以上）
# 子图3: 评分分布条形图（加数字标签）
# 子图4: 支付方式饼图
# 子图5: 各州订单量 Top10 水平条形图
# 子图6: 商品类别 Top10 水平条形图
plt.savefig("outputs/eda_distributions.png", dpi=150, bbox_inches="tight")
```

8.1.4 外键完整性检查

```
# LEFT JOIN + IS NULL 模式检查四组外键关系
# 结果：所有外键完整，无孤立记录
```

8.2 01_delay_analysis.ipynb — 延误分析

8.2.1 容忍度曲线

```
fig, ax = plt.subplots(figsize=(10, 6))
ax.plot(range(len(r)), r["br"].values, marker="o", color="crimson", linewidth=2)
ax.set_xticklabels(r["grp"].values, rotation=30)
ax.set_ylabel("差评率(%)")
ax.set_title("延误天数 vs 差评率（订单级去重）")
plt.savefig("../outputs/tolerance_curve.png", dpi=150, bbox_inches="tight")
```

解释：4 组（准时/1-3天/4-7天/8天+）差评率折线图，验证容忍度阈值约 3 天。

8.2.2 同品类评分差异图

```
fig, ax = plt.subplots(figsize=(10, 7))
vals = cat.head(15)["score_diff"].values
lbls = cat.head(15)["cat"].values
colors = ["crimson" if x < 0 else "seagreen" for x in vals]
ax.barh(lbls[::-1], vals[::-1], color=colors[::-1], edgecolor="white")
ax.axvline(x=0, color="black", linewidth=0.8)
plt.savefig("../outputs/category_delay_impact.png", dpi=150, bbox_inches="tight")
```

解释：Top 15 品类 score_diff 横向条形图。深红色=负面影响，竖线标注零点。

8.2.3 结论

```
print("1. 订单级去重后：延误显著拉低评分与差评率")
print("2. 差评率随延误天数递增，存在明显拐点")
print("3. 同品类控制后，延误仍显著负向影响")
```

九、分析报告.md — 分析报告

文件位置：分析报告.md

说明：面向阅读的最终分析报告，综合所有查询结果。包含 6 章：

章节	内容
一、项目概述	数据背景（10万+订单）、分析主线（物流效率与客户体验）、表结构
二、数据清洗与探索	缺失值处理、去重、外键完整性、U型评分分布、SP占40%
三、核心指标分析	GMV月度趋势（15万->110万）、品类Top5、支付方式、各州分布
四、RFM用户分层	高价值用户集中在东南部，单次购买占比高
五、物流延误影响分析	延误率6.77%、差评率6.8倍、容忍度阈值~3天、品类控制
六、结论	3条结论+3条业务建议（动态承诺/分层响应/品类差异化配送）

十、README.md — 项目说明

文件位置：README.md

说明：项目入口文档，包含： - 项目结构概览（模块清单、入口文件） - 运行指南 - **重要口径说明**：v_analysis_base vs v_order_delay 的适用场景 - 主线（延误容忍度阈值）与放弃项（同客户纵向对比因样本不足放弃） - 已知局限与改进方向（RFM区分度有限、可补 statsmodels 多元回归）

附录：数据流全景图

```
archive/原始CSV
    | setup_db.py
    v
DuckDB表 + 索引 + 基础视图 (v_order_summary, v_customer_metrics)
    | data_cleaning.py
    v
清洗视图层:
  v_valid_orders     -> 有效订单 (排除canceled/unavailable)
  v_unique_customers -> 去重客户 (ROW_NUMBER去重)
  v_order_delivery   -> 订单送达状态 (准时/延误标记)
  v_analysis_base    -> 明细宽表 (订单x商品x支付，可能多行)
  v_order_delay      -> 订单级去重 (一单一行，关键视图)
    |          |            |
    v          v            v
01_core_metrics  03_rfm.sql   04_delay_analysis.sql
02_funnel.sql    (RFM分层)    (延误专题:容忍度阈值+品类控制)
    |
    v
分析报告.md (综合所有分析结论)
```

关键口径

视图	粒度	用途
v_analysis_base	明细行级（可能多行/单）	GMV、品类结构、需要明细的 JOIN
v_order_delay	**订单级（一单一行）**	**延误率、差评率、容忍度曲线**

文件清单索引

文件	行数	类型	作用
setup_db.py	~130	Python	从CSV建DuckDB库+索引+基础视图
scripts/data_cleaning.py	~160	Python	创建5个清洗视图
scripts/01_core_metrics.sql	~60	SQL	6个核心指标查询
scripts/02_funnel.sql	~60	SQL	3个漏斗分析查询
scripts/03_rfm.sql	~70	SQL	5个RFM分层步骤
scripts/04_delay_analysis.sql	~80	SQL	5个延误专题查询
run_all_sql.py	~60	Python	一键执行所有SQL
notebooks/00_data_exploration.ipynb	~250	Notebook	数据探索EDA
notebooks/01_delay_analysis.ipynb	~150	Notebook	延误分析+可视化
分析报告.md	~300	Markdown	最终分析报告
README.md	~80	Markdown	项目入口说明